AI-Based Restoration of Degraded Images

SEMICON India Hackathon 2026 · KLA Problem Statement 01

TEAM DETAILS

Team name: *‹ fill in ›*

College / Institution: *‹ fill in ›*

Contact email: *‹ fill in ›*

Contact phone: *‹ fill in ›*

MEMBERS & ROLES

1. *‹ name ›* — *‹ role ›*
2. *‹ name ›* — *‹ role ›*
3. *‹ name ›* — *‹ role ›*
4. *‹ name ›* — *‹ role ›*

Repository: https://github.com/Deven1305/GrayScale

Deck generated from measured results only — no figure on any slide is estimated or invented.

Problem Statement Addressed

AI-Based Restoration of Degraded Images — speckle + Gaussian noise + 2× downsampling

WHY THIS MATTERS IN SEMICONDUCTOR FABS

Metrology and inspection tools are run at their throughput limit. Shorter dwell time per site means fewer collected electrons or photons, so images arrive noisy and spatially undersampled.

Restoring signal after acquisition recovers measurable detail without slowing the tool — the same wafer throughput, but usable images. That trade sits directly on fab economics: scan slower for image quality, or restore computationally and keep the tool moving.

WHAT THE DATA ACTUALLY IS — stated honestly

The released training set is 800 natural photographs (4 crops each = 3200 pairs), not semiconductor imagery. KLA's own materials additionally show scientific/microscopy content from a different release.

The test set is promised to be out-of-distribution. **That makes cross-origin generalisation the central technical challenge, not raw in-distribution accuracy.**

1 · SPECKLE NOISE

— Multiplicative, signal-dependent grain

— Pushes values beyond the true range

— Removed without blurring — blur destroys the detail we are paid to recover

2 · GAUSSIAN NOISE

— Additive, signal-independent

— Softens edges and fine structure

— Sharpness restored without ringing or invented texture

3 · 2× DOWNSAMPLING

— 256×256 → 128×128 in this release

— 512×512 → 256×256 in KLA's deck

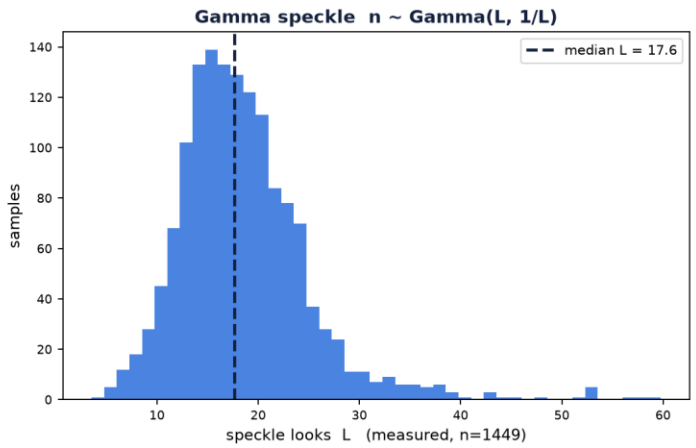
— Lost high-frequency detail reconstructed at exactly 2× the input

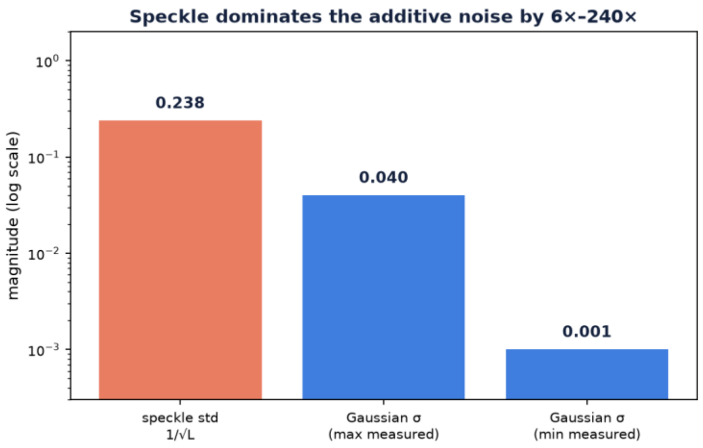
All three may be present simultaneously on one image — but not in equal measure. We measured speckle std ≈ 0.24 against additive σ of 0.001–0.04, so the design treats them by weight, not as three equal problems.

Idea Description

Build a despeckler first — the degradation is not three equal problems

KEY FINDING: we measured speckle std ≈ 0.24 against additive σ of 0.001–0.04 — multiplicative noise dominates by 6× to 240×.
Model capacity, loss design and augmentation all target it first.





Degradation	How the design addresses it
Speckle	Gamma-aware replica + log-domain unrolled prior; Charbonnier + MS-SSIM
Gaussian	Same denoising path; FFT + gradient loss restore edges without ringing
2× downsample	PixelShuffle(2) head on a bicubic residual anchor

MODEL CHOICE — NAFNet-w32

— No activation functions; LayerNorm generalises well out-of-distribution

— Best quality-per-FLOP among general restoration backbones

— Fully convolutional → both input sizes work unchanged

— SAFMN (~0.24 M) kept as the fast point on the Pareto curve

WHY NOT A GAN OR DIFFUSION

— The spec forbids “artificial patterns or ringing” — exactly what adversarial training produces

— GAN training costs PSNR and SSIM, two of the three scored metrics

— Decision recorded up front rather than discovered late

Proposed Solution

Architecture · training strategy · loss design · augmentation

INFERENCE PATH

Degraded input .npy float32 → **Reflect-pad** to mult. of 16 → **NAFNet-w32** at LR resolution → **PixelShuffle ×2** ICNR init → **+ bicubic anchor** residual → **Output** exactly 2×

Never clipped. Processing stays at LR resolution throughout — upsampling first would cost ~4× the compute for no quality gain.

TRAINING DATA FLOW

Clean HR source KLA GT + external → **× Gamma(L,1/L)** speckle at HR → **+ $N(0, \sigma^2)$** additive → **Cubic ×2 decimation** LAST — measured → **Paired (LR, HR)** float32

Order is fixed, not randomised — we measured it. Order randomisation is kept as an explicit OOD augmentation switch, evaluated rather than assumed.

TRAINING STRATEGY

— AdamW, lr 2e-4, cosine + warmup
— bf16 autocast (no GradScaler)
— EMA decay 0.999, evaluated with EMA
— Split by source_id = index // 4

COMPOSITE LOSS

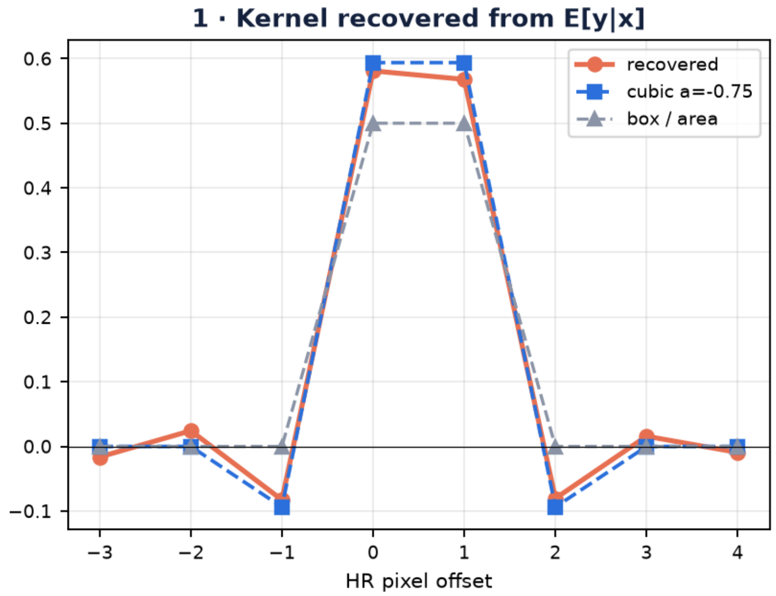
— 1.0 · Charbonnier
— 0.2 · (1 − MS-SSIM)
— 0.1 · FFT (L1 in frequency)
— 0.05 · gradient + 0.01 · VGG

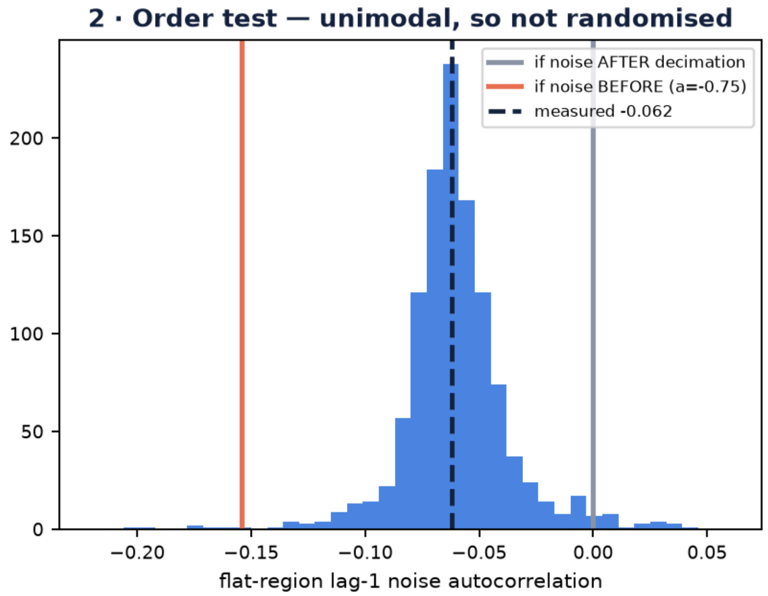
AUGMENTATION

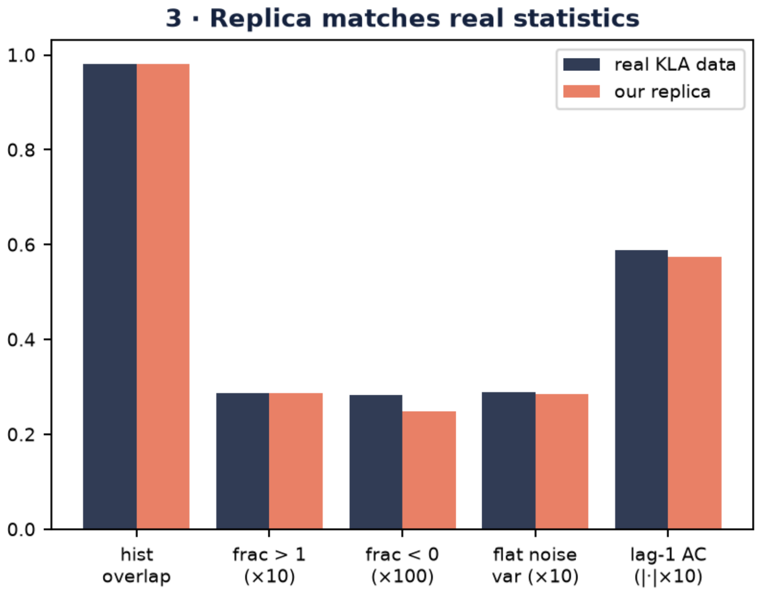
— 8-fold dihedral on paired crops
— L ~ 8–40, σ ~ log-uniform(1e-3, 4e-2)
— Kernel a ~ U(−0.75, −0.45)
— 50/50 KLA pairs vs external content

Innovation & Uniqueness

The degradation was reverse-engineered from the data, not assumed







1 · RECONSTRUCTED DEGRADATION

— No metadata ships with the dataset — we recovered the model from the pairs

— Kernel regressed directly from E[y|x]; box, area, Lanczos and strided decimation all excluded

— Replica hits 0.982 histogram overlap with the real data

2 · LOG-DOMAIN UNROLLED NET

— Built from KLA's own appendix reference (Monga et al. 2021) — a slide never presented

— log() turns multiplicative speckle additive, so HQS/ADMM machinery applies

— K analytic data-fidelity steps alternating with a learnt CNN prior

3 · SYNTHETIC OOD VALIDATION

— KLA withheld test ground truth — we manufacture it

— Validated replica applied to external corpora gives labelled OOD families

— Degradation-consistency check scores unlabelled test images with no GT

Results

Every number measured on held-out data. Nothing estimated.

RESTORATION QUALITY · 320 val images, held out BY SOURCE

Method	PSNR ↑	SSIM ↑	LPIPS ↓
Bicubic ×2 (floor)	23.067	0.5129	0.4425
BM3D + bicubic ×2	25.956	0.6527	0.5576
NAFNet-w48 (ours)	26.415	0.7333	0.2455

Beats every baseline on all three scored metrics: +3.35 dB / +0.220 SSIM / −0.197 LPIPS over bicubic, and +0.46 dB / +0.081 SSIM / −0.312 LPIPS over BM3D.

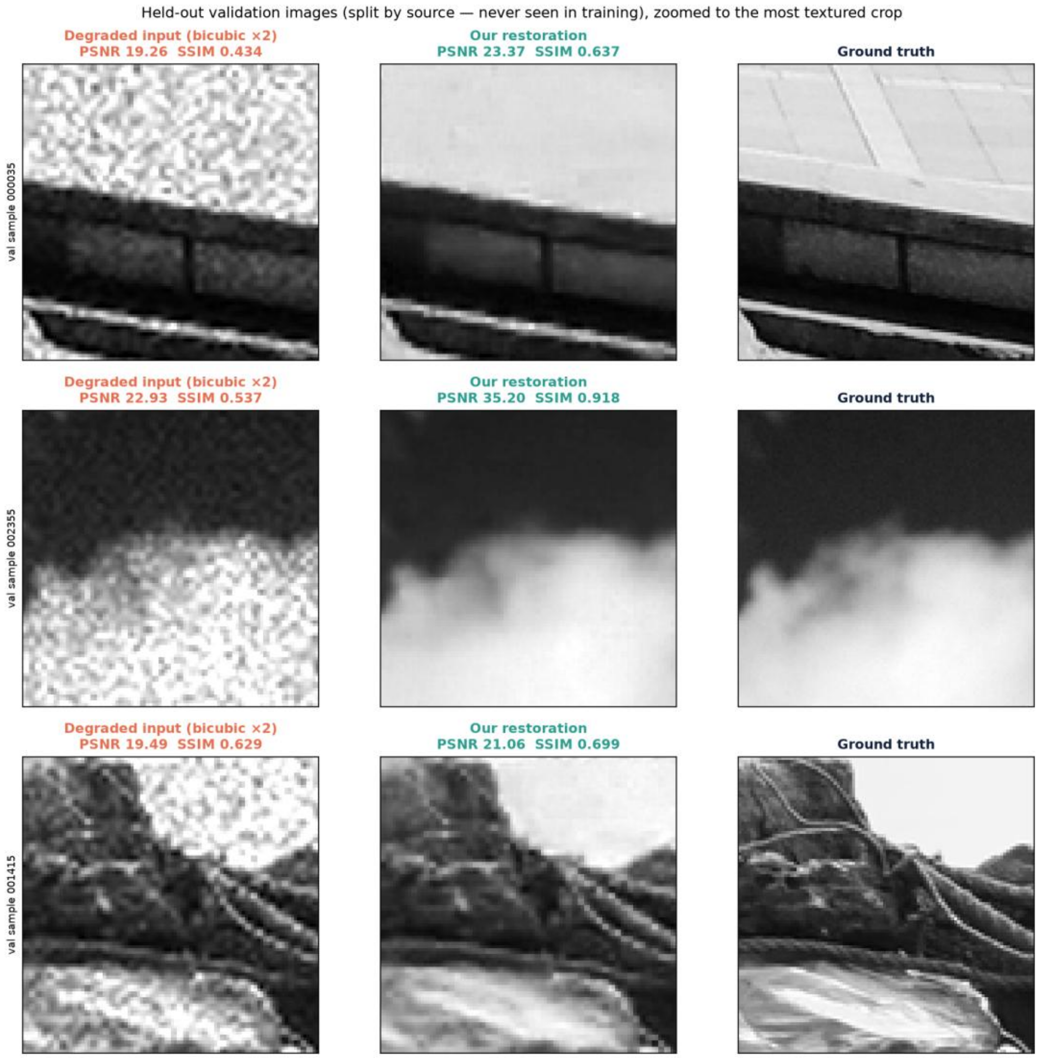
Note BM3D's LPIPS (0.558) is worse than plain bicubic (0.443): it buys PSNR by over-smoothing, which PSNR rewards and LPIPS punishes — the failure the spec warns against, quantified.

OUT-OF-DISTRIBUTION · synthetic GT from our own replica

Family	PSNR ↑	SSIM ↑	LPIPS ↓
Urban100 (buildings)	24.432	0.7750	0.2204
BSD100 (natural)	26.422	0.7506	0.2494
Set14	26.655	0.7643	0.2110

Urban100 drops 1.98 dB — a genuine OOD penalty on the buildings case KLA named. BSD100 and Set14 hold. Ground truth exists only because we reconstructed the degradation.

Degradation replica fidelity: 0.982 histogram overlap · frac>1 0.0287 vs 0.0288 · frac<0 0.00249 vs 0.00282 · noise lag-1 −0.0574 vs −0.0588



Held-out validation crops — never seen in training.

Technology & Feasibility

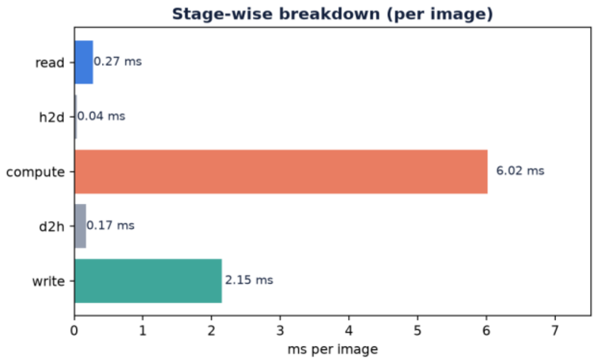
Stack, hardware, and the operating-point rule

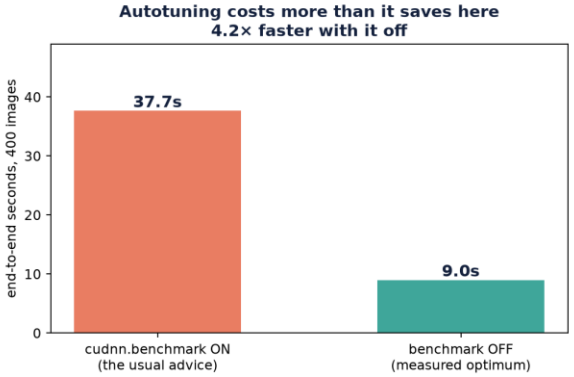
TECH STACK

Component	Choice
Framework	PyTorch 2.11.0 + CUDA 12.8
Training GPU	consumer GPU, 8 GB (recent architecture)
Benchmark target	NVIDIA H100 (KLA-side)
Precision	bf16 autocast (train) · fp16 (inference)
Environment	conda, Python 3.11.15, 115 pinned packages
Metrics	torchmetrics · lpips · pyiqa · bm3d

MEASURED PERFORMANCE · 400-image test set

Measure	Value
Model parameters	15.24 M
Weight file (fp16)	30.6 MB (8× smaller than ckpt)
Training time	40 epochs, ~1.6 h on an 8 GB GPU
Peak training VRAM	7.1 GB of 8 GB
Inference, per image	22.5 ms end-to-end
Full test set (400 img)	9.0 s · 44.4 img/s





WHAT THE MEASUREMENT CHANGED

Two defaults measured, then reversed.

cudnn autotuning costs 11.7 s of warmup for an ~8 % main-loop gain — a 2.4x end-to-end LOSS at this scale. And DataLoader workers are slower where they spawn, because each re-imports torch: 35.2 s with 4 workers vs 16.4 s with none.

Both are now platform-aware flags, not hardcoded, since a Linux H100 forks workers cheaply and may flip the answer back.

GitHub & Video Link

Repository is mandatory; video optional

GITHUB REPOSITORY (mandatory)

https://github.com/Deven1305/GrayScale

Visibility must be set to PUBLIC before submission — a private repository cannot be benchmarked, and unscored submissions cannot win.

DEMONSTRATION VIDEO (optional, recommended)

‹ TO BE ADDED ›

REQUIRED REPOSITORY CONTENTS

#	Item	Status
1	README.md — clone-and-run instructions	delivered
2	Evaluation script (standalone .py, dir → dir)	delivered
3	Training script (train.py, config-driven)	delivered
4	Trained model weights (10 MB fp16)	delivered
5	Restored test outputs (400 images)	delivered
6	requirements.txt (pip freeze)	delivered

References

KLA-cited sources, methods, and datasets

CITED BY KLA IN THE PROBLEM-STATEMENT MATERIALS

[1] T. Kumar, R. Brennan, A. Mileo, M. Bendechache. “Image Data Augmentation Approaches: A Comprehensive Survey and Future Directions.” IEEE Access, vol. 12, 2024.

[2] L. Zhai, Y. Wang, S. Cui, Y. Zhou. “A Comprehensive Review of Deep Learning-Based Real-World Image Restoration.” IEEE Access, vol. 11, pp. 21049–21067, 2023.

[3] J. Terven, D. M. Cordova-Esparza, J. A. Romero-González et al. “A Comprehensive Survey of Loss Functions and Metrics in Deep Learning.” Artificial Intelligence Review 58, 195, 2025.

[4] V. Monga, Y. Li, Y. C. Eldar. “Algorithm Unrolling: Interpretable, Efficient Deep Learning for Signal and Image Processing.” IEEE Signal Processing Magazine, vol. 38, no. 2, pp. 18–44, 2021.

OUR OWN ANALYSIS

— docs/forensics_report.md — degradation model reconstructed from the paired data

— 25 standalone, re-runnable forensics scripts

— All figures on this deck generated from measured data

METHODS

[5] L. Chen, X. Chu, X. Zhang, J. Sun. “Simple Baselines for Image Restoration.” ECCV 2022. (NAFNet)

[6] K. Zhang, J. Liang, L. Van Gool, R. Timofte. “Designing a Practical Degradation Model for Deep Blind Image Super-Resolution.” ICCV 2021. (BSRGAN)

[7] K. Zhang, L. Van Gool, R. Timofte. “Deep Unfolding Network for Image Super-Resolution.” CVPR 2020. (USRNet)

[8] L. Sun, J. Dong, J. Tang, J. Pan. “Spatially-Adaptive Feature Modulation for Efficient Image Super-Resolution.” ICCV 2023. (SAFMN)

DATASETS & TOOLS

[9] E. Agustsson, R. Timofte. “NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study.” CVPRW 2017. (DIV2K)

[10] J.-B. Huang, A. Singh, N. Ahuja. “Single Image Super-Resolution from Transformed Self-Exemplars.” CVPR 2015. (Urban100)

[11] M. Cimpoi et al. “Describing Textures in the Wild.” CVPR 2014. (DTD)

[12] K. Dabov et al. “Image Denoising by Sparse 3-D Transform-Domain Collaborative Filtering.” IEEE TIP, 2007. (BM3D)

[13] R. Zhang et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric.” CVPR 2018. (LPIPS)